High-Level Design (HLD)

1. Purpose and Scope

The application is a multi-vendor bespoke tailoring marketplace. Customers discover tailors, maintain garment measurements, commission custom work, and track orders. Tailors manage shop information, designs, customers, and order production. Administrators manage platform operations and shop approvals.

This document describes the implemented system boundaries and major runtime interactions. It is based on the current repository; it is not a deployment blueprint for a production environment.

2. System Context

```
Customers / Tailors / Administrators
                 |
                 v
        Web browser application
         React + TypeScript + Vite
                 |
       JSON API requests (/api/*)
                 |
                 v
          FastAPI application
  Authentication, role checks, business services
                 |
         SQLAlchemy data access
                 |
                 v
       PostgreSQL (configured target)
       SQLite (development fallback)

Payment integration: MOCK_SANDBOX by default
```

The frontend and backend are separate applications. The browser calls the backend API using the configured API base URL. The API is mounted under `/api`; the backend also mounts the router at the root for backward compatibility.

3. Major Components

Component	Responsibility
React frontend	Public marketplace, registration/login, role dashboards, language/theme controls, and API calls.
Shared frontend services	Build API requests, attach the stored bearer token, apply a request timeout, and normalize API errors.
FastAPI API	Expose authentication, discovery, customer, measurement, order, delivery, admin, payment, review, health, and international endpoints.
Authentication and dependencies	Validate JWT bearer tokens, load active users, and enforce customer/tailor/admin roles.
Backend services	Apply domain rules for accounts, discovery, measurements, orders, payments, reviews, administration, and health.
SQLAlchemy models and database	Persist users, shops, designs, measurements, orders, delivery, payments, and reviews. PostgreSQL is the configured target; local SQLite fallback is present when PostgreSQL cannot be reached.
Static file mount	Serve uploaded order-reference files from the backend static directory.

4. Main User Workflows

Customer commission

1. The customer signs in and receives a bearer token.
2. The customer browses shops/designs and creates or selects a garment measurement profile.
3. The frontend submits an order with its items, customization notes, measurement references, currency, and delivery address.
4. The backend validates and stores the order and its related records.
5. The tailor advances production milestones; the customer views order and delivery status.
6. Payment records can be created and confirmed. The configured default is a mock sandbox, not a live payment processor.

Tailor operations

Tailors use a role-protected dashboard to manage their storefront, designs, customer records, and production orders. API role checks also protect tailor-specific data; frontend route protection is not the only authorization boundary.

Administration

Administrators use a separate protected dashboard and admin endpoints for platform metrics and operational management, including shop approval workflows.

5. Cross-Cutting Capabilities

- **Access control:** JWT bearer authentication plus server-side role and ownership checks.
- **Localization:** English, Telugu, and Hindi translation catalogs, language selection persistence, and English fallback for missing keys.
- **Theme:** Dark/light theme selection persisted in browser storage.
- **International support:** Country and currency catalogs, address/phone validation, and delivery-fee configuration.
- **Operational checks:** Health endpoints and startup initialization/seeding for local demonstration data.
- **Error handling:** The frontend API client applies a 10-second request timeout, extracts API error details, and clears stored authentication data on unauthorized responses.

6. Runtime and Data

The backend initializes SQLAlchemy metadata during application startup, synchronizes selected existing columns, and seeds demo accounts, shops, measurement templates, and designs. PostgreSQL is the configured database target. If its connection is unavailable, the current code falls back to a local SQLite file for development.

The repository contains Alembic configuration, but production migration execution and deployment topology are not established by this HLD. The current payment provider defaults to `MOCK_SANDBOX`; production provider credentials, callbacks, and webhook handling must be completed and verified before accepting real payments.

7. Known Release Gaps

- Configure production secrets and database settings; remove development defaults.
- Establish and test a production migration and deployment process.
- Integrate and test a real payment provider, including server-verified callbacks/webhooks.
- Run the full test suite in a clean repeatable environment and validate operations, monitoring, backups, and recovery.

8. Technology Summary

- Frontend: React, TypeScript, React Router, Vite, and Tailwind CSS.

- Backend: Python, FastAPI, Pydantic, and SQLAlchemy.
- Database: PostgreSQL target; SQLite fallback for development.
- API contract: JSON over HTTP; JWT bearer authentication for protected endpoints.